SKILLFORGE

Project Overview & Team Implementation Handbook

CS1207 Object-Oriented Programming Lab

Item	Working baseline
Project	Student Project & Team Collaboration Platform
Team	4 members: Charan, Ephraim, Balanand, Manish
Working start	12 October 2026
Submission target	31 October 2026
Status	Working reference; proposed decisions are not automatically approved

Purpose. A shared reference for project scope, requirements, workflows, architecture, technology choices, team responsibilities, testing, deployment and unresolved decisions.

Status rule. Team answers and recommendations remain proposed until explicitly approved. Unknown faculty requirements, hosting choices and implementation details are not invented.

Prepared from SkillForge discussions through 11 October 2026.

Contents

- 1. Executive summary and scope
- 2. Functional requirements
- 3. Decision status and approval protocol
- 4. Technology stack
- 5. Architecture and package layout
- 6. Class model and database design
- 7. Core workflows
- 8. Permissions and security
- 9. Business rules and calculations
- 10. Role-specific delivery pages
- 11. GitHub and integration workflow
- 12. Testing and traceability
- 13. Non-functional requirements
- 14. Deployment and operations
- 15. Timeline
- 16. Risks and open decisions
- 17. SRS structure
- 18. Definition of done

Status vocabulary

Label	Meaning
Proposed	Recorded team answer or recommendation; not formally approved.
Open	Decision, external fact or policy detail still unresolved.
Confirmed context	Fact explicitly established in project discussion.
Implementation requirement	Expected behavior for the proposed v1 baseline; must be signed off and tested.

1. Executive summary and project scope

SkillForge is a student-focused web application for student profiles, skill-based teammate discovery, academic/hackathon project coordination, invitations, task assignment, deadline management, progress tracking and portfolio showcase. The team has four members and aims to submit on 31 October 2026.

Goals

- Students maintain profiles containing skills, interests, achievements and portfolio items.
- Signed-in students search/discover profiles; matching shows coverage of a project's required skills.
- Project owners create projects, invite students, manage membership and appoint leaders.
- Project teams create tasks, assign current members, set deadlines and update status.
- Project progress is calculated from completed tasks; archived projects preserve read-only history.
- Demonstrate meaningful Java OOP, relational data design, secure workflows, tests and deployment.

Out of scope unless explicitly approved

- Chat/video, AI matching, social login, file uploads, faculty grading dashboard, gamification and native mobile app.
- Public browsing of full profiles or full project details.
- Restoring archived projects in v1; archived projects remain permanently read-only in the current proposal.
- Email verification and automated password reset in v1; current proposal uses a documented manual recovery process.

Constraints and known gaps

- Online deployment is required, but provider and deployment method remain open.
- Faculty rubric and mandatory UML diagrams are not available yet.
- All proposed decisions require explicit team approval before they become final.

2. Functional requirements

ID	Requirement heading	Expected behavior / acceptance direction
FR-01	Student account access	Register with valid email, unique username and password; sign in by email or username; logout; protect authenticated routes.
FR-02	Manage student profile	Owner edits own profile, skills, interests, achievements and portfolio.
FR-03	Search student profiles	Signed-in students discover profiles according to visibility rules.
FR-04	Skill-based teammate matching	Coverage score based on distinct required skills; unavailable if no required skills; deterministic tie-break.
FR-05	Create/manage projects	Create project with title, description, required skills, deadline and max team size; role-based edits.
FR-06	Invitations and membership	Owner/leader invites; recipient accepts/declines; 7-day expiry; duplicate and capacity rules.
FR-07	Task assignment and deadlines	Members create tasks and assign current members; task includes due date and status.
FR-08	Task status and progress	To Do → In Progress → Done; progress from completed/total tasks; unavailable for zero tasks.
FR-09	Portfolio showcase	Text descriptions, achievements, project links and external work-sample URLs; no uploads.

These are proposed requirement headings, not yet complete normative SRS statements. Charan should write precise 'The system shall…' requirements with acceptance criteria and traceability.

Current proposed exclusions

- No chat, video, AI matching, social login, file uploads, faculty dashboard, gamification or native mobile app unless scope changes.
- No separate Overdue task status; overdue is derived from due date and task status.
- No archive restoration in v1.

3. Decision status and approval protocol

Area	Recorded proposal	Status / caveat
Eligibility	Students only; any email provider; self-declared student status; no verification.	Proposed; student status is not independently verified.
Identity	Unique username; login by email or username plus password.	Proposed; normalization and username-change rules open.
Database	MySQL; Spring Data JPA/Hibernate; Hibernate schema create/update during development.	Proposed; production migration approach open.
Backend/UI	Java 21 LTS, Spring Boot, Spring MVC, Thymeleaf, Maven.	Proposed stack.
Tools	GitHub; all four need access; VS Code.	Proposed; verify repository and access.
Architecture	Layered monolith: Controller → Service → Repository → MySQL.	Proposed.
Security	Server-side sessions, logout invalidation, CSRF, BCrypt, server-side authorization.	Proposed; implementation and testing still required.
Projects	Invitation-only, 7-day expiry, capacity rechecked at acceptance.	Proposed; concurrency test required.
Ownership	Existing member must explicitly accept; transfer atomic.	Proposed; timeout/owner unavailability open.
Archive	Owner archives; permanent read-only in v1.	Proposed.
Hosting	Online deployment required.	Open provider, Java runtime, MySQL, URL.
Faculty	Meaningful OOP; UML unknown; rubric unavailable.	Open.
Submission	Source, SRS, database materials, tests and demo expected.	Proposed pending official checklist.

Approval protocol

1. Record each decision as Approved, Rejected or Deferred, with date, rationale and responsible person.
2. If a choice changes, record old/new decision and impacted FR, use-case, diagram and test IDs.
3. Do not mark a feature complete because it appears in this handbook; verify implementation and test results.
4. Faculty instructions override proposals if they conflict.
5. Keep unresolved questions visible rather than silently choosing the easiest implementation.

4. Technology stack

Layer	Proposed technology	Notes
Language/runtime	Java 21 LTS	Verify JDK installed on all machines and compatible with hosting.
Backend	Spring Boot	Integrates MVC, Security, JPA and test tooling.
Web	Spring MVC	Server-rendered form workflows and controllers.
Frontend	Thymeleaf + HTML/CSS/JavaScript	No separate SPA in current proposal.
Database	MySQL	Relational persistence.
ORM	Spring Data JPA + Hibernate	Entity mappings and repository interfaces.
Build	Maven	One checked-in pom.xml and compatible dependency management.
Security	Spring Security	Authentication, BCrypt, sessions, CSRF and route protection.
Tests	JUnit 5, Spring Boot Test, MockMvc where useful	Unit, service, integration and authorization tests.
Source control	GitHub	Branches/PRs; never commit secrets.
IDE	VS Code	Portable project setup; IDE choice does not alter build.
Hosting	Not selected	Must support Java app and compatible MySQL connectivity.

Proposed dependency categories

- Spring Boot Web, Thymeleaf, Spring Data JPA, MySQL JDBC driver, Spring Security, Validation, Spring Boot Test and Spring Security Test.
- For production, select a controlled migration approach such as Flyway after team approval; do not rely on Hibernate automatic update as the only production schema strategy.
- Choose compatible stable versions through Spring Boot dependency management; verify Java 21 compatibility before committing pom.xml.

5. Architecture and package layout

Use a layered monolith: it is simpler to deploy and test for a four-person team while keeping business rules separate from web pages and persistence.

```
Browser: Thymeleaf-rendered HTML forms
          |
          v
Controller layer: routes, form binding, validation feedback
          |
          v
Service layer: permissions, matching, workflows, transactions
          |
          v
Repository layer: Spring Data JPA interfaces
          |
          v
Entity/model layer: Student, Project, Task, etc.
          |
          v
MySQL relational database
```

Suggested package layout

```
com.skillforge
■■■ config/       SecurityConfig and application config
■■■ controller/   Auth, Profile, Project, Task controllers
■■■ dto/          Form/request DTOs; don't bind sensitive fields
■■■ model/        JPA entities and enums
■■■ repository/   Spring Data repositories
■■■ service/      Account, Profile, Project, Task services
■■■ security/     UserDetailsService and authorization service
■■■ exception/    Domain exceptions and safe error handling
■■■ SkillForgeApplication.java

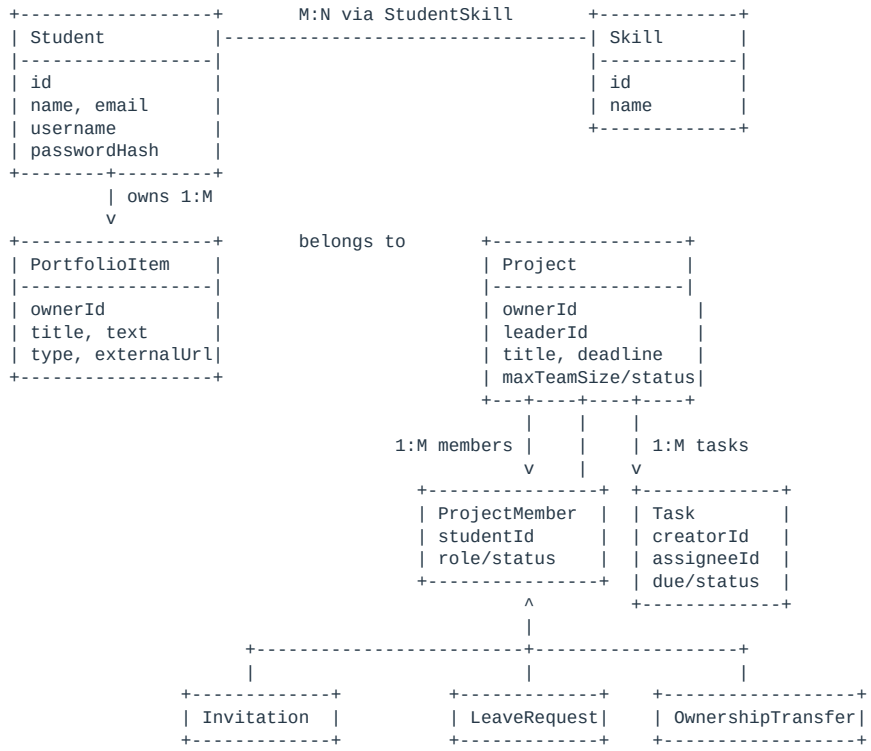
src/main/resources/
■■■ templates/    Thymeleaf pages and fragments
■■■ static/css/   Styles
■■■ static/js/    Small UI behaviors
■■■ application.properties (no production secrets)
```

Layer rules

- Controllers stay thin; validate and delegate to services.
- Services enforce business invariants and permissions; hiding a button is not authorization.
- Repositories handle persistence queries, not high-level workflow decisions.
- Use DTOs to prevent client-supplied roles, owner IDs, hashes and audit fields.
- Use transactions for invitation acceptance, capacity checks and ownership transfer.
- Keep matching calculations independently unit-testable.

6. Class model and database design

This is a proposed UML-style model derived from the scope, not a claim that classes already exist. Refine it against faculty expectations and actual code.



Core entities and constraints

Entity/table	Key fields and constraints
students	id PK; normalized email UNIQUE; normalized username UNIQUE; password_hash; status; timestamps.
skills / student_skills	Skill normalized_name UNIQUE; bridge has FKs and UNIQUE(student_id, skill_id).
portfolio_items	owner_id FK; title, description, type, external_url; timestamps.
projects / project_skills	owner_id FK; nullable leader_id FK; title, description, deadline, max_team_size, status; distinct required skills.
project_members	project_id, student_id, role, membership_status, joined_at; enforce one active membership.
invitations	project, sender, recipient, status, sent_at, expires_at; prevent duplicate pending invite.
leave_requests	project, requester, status, reviewer and timestamps; requester cannot self-approve.
ownership_transfers	project, from_owner, to_owner, status, accepted/completed timestamps.
tasks	project, creator, assignee, title, description, due_date, status, timestamps.

7. Core workflows

7.1 Registration and login

```
Registration page
 -> validate email, username and password
 -> enforce normalized unique email/username
 -> store BCrypt hash (no verification in v1 proposal)
 -> sign in by email OR username + password
 -> create server-side session
 -> logout invalidates session
```

7.2 Search and matching

```
Signed-in student searches profiles/projects
 -> apply visibility rules
 -> open permitted profile/project
 -> calculate required-skill coverage
 -> sort score descending, username ascending
 -> if no required skills: show 'Matching unavailable'
```

7.3 Project creation and invitations

```
Create project -> creator becomes owner and current member
 -> set title, description, required skills, deadline, team limit
 -> owner/leader sends invitation
 -> recipient accepts/declines within 7 days
 -> acceptance transaction rechecks recipient, expiry, project state, capacity
 -> create membership and resolve invite OR reject with clear error
```

7.4 Tasks and progress

```
Current member creates task -> assign current project member
 -> To Do -> In Progress -> Done
 -> creator/assignee edit details; owner/leader can edit any task
 -> progress = Done tasks / all tasks * 100
 -> zero tasks = 'Progress unavailable'
 -> overdue is derived from due date and status
```

7.5 Leave, ownership transfer and archive

- Member submits leave request; owner approves/rejects; no self-approval.
- Owner must transfer ownership before leaving. Target must already be a current member and explicitly accept.
- Rejected/ignored transfer leaves current ownership unchanged; another eligible member may be proposed.
- Leader waits until owner appoints replacement before leaving under the current proposal.
- Only owner archives; archived project is permanently read-only in v1. Former members may view history under the proposal; exact visible fields remain open.

8. Roles, permissions and security

Action	Non-member	Member	Leader	Owner
View limited discovery summary	Yes, signed in	Yes	Yes	Yes
View full active project details	No	Yes	Yes	Yes
Edit own profile/portfolio	Own only	Own only	Own only	Own only
Edit project details	No	No	Yes	Yes
Send invitations	No	No	Yes	Yes
Create task/update status	No	Yes	Yes	Yes
Edit task details	No	Creator/assignee	Any task	Any task
Approve member leave	No	No	No	Yes, not own request
Appoint/replace leader	No	No	No	Yes
Archive project	No	No	No	Yes
Ownership transfer	No	No	No	Initiates; target accepts

Security baseline

- Enforce authentication and authorization on server routes and again in service operations.
- Resolve current Student from the authenticated server context; never trust submitted IDs/roles as authority.
- BCrypt password hashes only; never log passwords or store plaintext.
- CSRF protection for state-changing forms; invalidate server session on logout.
- Keep DB credentials and hosting secrets out of GitHub; use environment/secret configuration.
- Validate and normalize user input; safely escape user text; validate external URL schemes.
- Transactions for capacity checks, invitation acceptance, ownership transfer and archive transitions.
- Audit important events (archive, transfer, leader changes, invitations and leave requests) without secrets.

9. Business rules and calculations

Matching coverage

```
coverage % = (distinct required skills the student has / total distinct required skills) * 100
```

- Deduplicate skills before calculating.
- No required skills means matching unavailable, not divide by zero.
- Sort coverage descending, then normalized username alphabetically (proposed tie-break).
- The score measures required-skill coverage only; it does not measure personality, availability or overall suitability.
- No skill proficiency levels in v1 proposal: a skill is present or absent.

Project progress

```
progress % = (number of Done tasks / total project tasks) * 100
```

- Zero tasks means progress unavailable, not 0%.
- Overdue is derived if due date has passed and status is not Done; define timezone/date boundary consistently.
- Task statuses currently proposed: To Do, In Progress, Done; no Cancelled status.
- Archived projects keep their state and reject modifications.

Core invariants

1. Normalized email and username unique.
2. Project owner is a current member; leader is a current member.
3. Task assignee must be a current member of the task's project.
4. Only intended recipient may accept an unexpired invitation for an active project with capacity.
5. Only owner archives, appoints/replaces leader and approves member leave under current proposal.
6. Ownership transfer completes only after eligible current member accepts.
7. Authorization is server-side; archived projects are read-only.

10. Team responsibilities and delivery pages

All four members contribute to testing and implementation. The role assignment below identifies primary accountability and handoffs; coordinate shared interfaces before parallel coding.

Shared team rules

- Agree entity names, enums, package names, routes, DTO fields and service contracts before parallel work.
- Use feature branches and review changes before merging to main.
- Keep README build/run/test instructions current; all four should be able to clone and run the app.
- Never commit secrets or production data; freeze v1 scope and defer extras.
- Each owner provides tests and a handoff note, not just code.

10A. Charan — Coordinator, SRS lead and test coordination

Primary outcome: keep decisions, requirements, diagrams, tests and delivery documentation consistent.

Responsibilities

- Maintain decision register with Approved/Rejected/Deferred, date, owner and rationale.
- Own SRS using faculty structure: Introduction; Overall Description; Specific Requirements; Non-functional Requirements; Other Requirements; Activity Log.
- Turn FR-01…FR-09 into testable 'shall' requirements after sign-off.
- Maintain use-case IDs, test IDs and requirement-to-test traceability.
- Obtain faculty rubric, mandatory diagrams and exact deliverables.
- Coordinate test plan, execution evidence, defect triage, regression and demo checklist.
- Keep SRS factual to actual implementation; record changes and known limitations.

Tools / technologies

- Faculty SRS template and document editor.
- GitHub Issues/Projects or agreed task board.
- JUnit/Spring test results supplied by developers; spreadsheet/checklist for manual evidence.
- PlantUML/diagrams.net if permitted by faculty.

Deliverables and done criteria

Deliverable	Done when
Decision register	Every choice has status, date, owner, rationale; blockers visible.
SRS	Faculty structure followed; requirements unique and testable.
Traceability	Every FR maps to use case, design component, test ID and result.
Test evidence	Core happy paths and edge/security tests have recorded outcomes.
Release handoff	Submission checklist, known limitations, run/deploy instructions and demo scenario ready.

Dependencies

- Needs actual implementation facts from all teammates.
- Do not invent UML mandates before faculty confirmation.
- First day: ask for rubric; confirm FR IDs; set test ID convention; create SRS/traceability skeleton.

10B. Ephraim — Frontend/UI

Primary outcome: implement clear Thymeleaf pages and forms for agreed workflows, without bypassing server permissions.

Template/page	Purpose
auth/register.html	Name, email, username, password; validation errors.
auth/login.html	One identifier field for email or username plus password.
dashboard.html	Signed-in landing page and navigation.
profile/view.html	Permitted profile, skills, achievements and portfolio display.
profile/edit.html	Owner-only profile editing.
students/search.html	Search/discover students and display matching coverage.
projects/list.html	Limited discovery summaries for non-members.
projects/detail.html	Member-only full details, team, tasks and progress.
projects/form.html	Create/edit project; owner-only archive action.
projects/invitations.html	Incoming/outgoing invites, status, expiry, accept/decline.
projects/members.html	Members/roles, leave requests and authorized controls.
tasks/list.html	Task status, assignee, due date and derived overdue.
tasks/form.html	Create/edit tasks; backend controls authorization.
portfolio/list.html + form.html	Signed-in viewing; owner-only create/edit/remove text and links.

Technology and rules

- Thymeleaf, HTML/CSS/JavaScript; reusable fragments for navigation, messages and form errors.
- CSRF tokens on state-changing forms; server-side validation messages.
- Semantic HTML, labelled fields, visible focus, keyboard usability, readable contrast and basic narrow-screen support.
- Escape user text and render external links safely; no upload feature.
- Do not rely on hidden buttons for security; service layer is authoritative.

Deliverables

- Templates/styles for agreed pages, wired to agreed routes/DTOs.
- Empty, success, validation-error and unauthorized states for core flows.
- Page test notes/screenshots if required; PR plus handoff of routes/forms and assumptions.

Dependencies

Needs route names, DTO fields and view attributes from Balanand/Manish; approved visibility and role rules; runnable application skeleton and security/Thymeleaf integration.

10C. Balanand — OOP/core logic

Primary outcome: implement testable domain behavior and business rules in Java services/entities, using meaningful OOP without artificial inheritance.

Responsibilities

- Model Student, Skill, Project, ProjectMember, Invitation, LeaveRequest, OwnershipTransfer, Task and PortfolioItem.
- Implement matching coverage/tie-break, project authorization, invitation expiry/duplicates/capacity, leave and ownership transitions.
- Implement task statuses, overdue calculation, progress and archive rules.
- Use domain exceptions/results for invalid states; coordinate transaction boundaries with Manish.
- Write unit tests for calculations/state rules and service tests for unauthorized actions.

OOP guidance

- Encapsulation: private fields, controlled methods and clear invariants.
- Abstraction: use interfaces where they clarify responsibilities, not as decoration.
- Polymorphism: only where a real interchangeable behavior exists and it helps the design.
- Inheritance: do not create artificial base classes just to tick a rubric box; prefer composition unless a genuine is-a relationship exists.
- Use enums for roles/statuses when appropriate.

Illustrative service contracts

```
MatchingService.calculateCoverage(student, project)
ProjectService.createProject(currentStudent, form)
InvitationService.sendInvitation(currentStudent, projectId, recipientId)
InvitationService.acceptInvitation(currentStudent, invitationId)
MembershipService.requestLeave(currentStudent, projectId)
OwnershipTransferService.propose(currentOwner, projectId, targetMemberId)
OwnershipTransferService.accept(targetMember, transferId)
TaskService.createTask(currentMember, projectId, form)
TaskService.updateStatus(currentMember, taskId, newStatus)
ProjectAuthorizationService.requireCanEditProject(currentStudent, project)
```

Deliverables and dependencies

- Services/domain code, unit tests, unauthorized-action tests, clear method contracts and PR handoff.
- Needs approved permission matrix, entity/enums/repository contracts from Manish, form expectations from Ephraim and test IDs from Charan.

10D. Manish — Database and integration

Primary outcome: provide reliable MySQL persistence, integrate the app, maintain reproducible builds, and make transactions/data constraints match the business rules.

Responsibilities

- Set up MySQL and local connection configuration.
- Define JPA entity mappings and repository interfaces with Balanand.
- Enforce normalized unique email/username, foreign keys, skill uniqueness and indexes.
- Implement/coordinate transactional invitation acceptance and ownership transfer; prevent capacity race conditions.
- Define development schema strategy and controlled production migration approach.
- Integrate controllers/services/templates and resolve build/configuration conflicts.
- Keep secrets out of GitHub; prepare and test database export/restore.
- Document clean-clone setup and final deployment/database connection.

Integrity/transaction rules

- Unique constraints at database and application levels.
- Unique (student_id, skill_id) and (project_id, skill_id).
- Prevent duplicate active membership and pending invitations using constraints/indexes plus transactional logic.
- Invitation acceptance rechecks project state, recipient, expiry and capacity inside transaction.
- Ownership transfer rechecks target eligibility and updates owner/roles atomically.
- Test concurrency/last-seat behavior; do not rely only on a pre-transaction count.
- Hibernate update is not the only production schema strategy.

Deliverables and dependencies

- App connects to MySQL from a clean clone; entity/repository mappings match approved model.
- Documented constraints, indexes, migration plan and tested backup/restore procedure.
- Integration tests for persistence, uniqueness, capacity and transfer.
- Needs final schema/enums/invariants from Balanand, hosting choice, requirements/tests from Charan and route/form contracts from Ephraim.

11. GitHub and integration workflow

1. Create shared GitHub repository and verify each teammate can clone and push a branch.
2. Use descriptive feature branches and pull requests or another agreed review step.
3. Commit small logical changes; do not commit .env, credentials, personal-data dumps or build output.
4. Keep README with JDK, Maven, MySQL setup, configuration, run/test commands and deployment notes.
5. Agree package/entity/enum names, routes, DTO fields, service signatures, error conventions and current-user access before parallel work.
6. Resolve merge conflicts with the relevant code owner; integrate and build daily.

Suggested test ID prefixes

Prefix	Scope	Example
AUTH	Registration/login/session	AUTH-02 duplicate username rejected
PROF	Profile/portfolio	PROF-01 user edits own profile
MATCH	Search/matching	MATCH-01 no required skills unavailable
PROJ	Project/roles/archive	PROJ-01 member cannot archive
INV	Invitations/capacity	INV-03 concurrent acceptances respect capacity
TASK	Task/progress	TASK-02 zero-task progress unavailable
SEC	Cross-cutting security	SEC-01 logged-out session denied
DEP	Deployment/operations	DEP-01 database restore succeeds

12. Testing and traceability

Test alongside implementation. A test is passed only when executed against the current build and its result is recorded.

Test ID	Scenario	Expected result
AUTH-01	Valid registration	Account created; password hashed.
AUTH-02	Duplicate normalized email/username	Rejected; no duplicate record.
AUTH-03	Login with email or username	Both approved identifiers work.
SEC-01	Protected route while logged out	Denied/redirected; no protected data.
SEC-02	Reuse session after logout	Old session no longer authorizes.
PROF-01	Edit another student's profile	Denied.
MATCH-01	No required skills	Matching unavailable; no divide-by-zero.
MATCH-02	Equal match scores	Username tie-break if approved.
INV-01	Wrong recipient accepts invitation	Rejected.
INV-02	Expired invitation accepted	Rejected.
INV-03	Two users compete for last seat	Capacity never exceeded.
INV-04	Duplicate pending invite	Prevented/rejected.
PROJ-01	Member edits project or archives	Denied.
PROJ-02	Leader edits details/sends invite	Allowed on active project if policy approved.
PROJ-03	Archived project modification	Rejected; history preserved.
OWN-01	Owner leaves without completed transfer	Rejected.
OWN-02	Valid accepted ownership transfer	Atomic update; old owner remains member unless separately leaving.
OWN-03	Rejected/ineligible transfer	Current owner retained; no partial update.
MEM-01	Requester approves own leave	Rejected.
TASK-01	Assign task to non-member	Rejected.
TASK-02	Zero project tasks	Progress unavailable.
TASK-03	Task overdue then marked Done	Derived overdue indicator updates.
SEC-03	Tampered owner/role ID in form	No privilege escalation.
DEP-01	Restore database backup	Documented restore succeeds.

Traceability template

FR ID	Use case	Design component	Page/controller/repository	Test ID	Result
FR-01	UC-01	Student/AccountService	AuthController/StudentRepository	AUTH-01…03	Not run
FR-04	UC-04	MatchingService	Search controller/repositories	MATCH-01…02	Not run
FR-06	UC-06	Invitation/ProjectService	InvitationController/InvitationRepository	INV-01…04	Not run
FR-08	UC-08	Task/ProgressService	TaskController/TaskRepository	TASK-02…03	Not run

Mappings above are illustrative. Charan should expand them to every final FR/use case and actual code component.

13. Non-functional requirements

ID	Proposed quality requirement	Verification
NFR-01	Desktop-first; basic small-screen usability.	Manual desktop/narrow viewport review.
NFR-02	Semantic HTML, labels, keyboard usability, readable contrast.	Keyboard and basic accessibility walkthrough.
NFR-03	Current Chrome/Edge; Firefox where possible.	Run critical flows in selected browsers.
NFR-04	Responsive core pages at demo scale.	Use representative data; record observed results.
NFR-05	Server-side validation/authorization.	Negative role/tampered request tests.
NFR-06	BCrypt, CSRF, secure sessions.	Config review and security tests.
NFR-07	Secrets not committed.	Repository/configuration scan.
NFR-08	Manual database backup/restore documented.	Test restore before demo.
NFR-09	Reliable during planned demonstration; no formal uptime guarantee.	Pre-demo smoke test.
NFR-10	Audit ownership transfer and archive actions.	Test audit records and sensitive-data exclusions.

Avoid unsupported numerical response-time or uptime claims unless the team defines measurable targets and can test them. The current performance proposal is qualitative and tied to demo scale.

14. Deployment and operations

Online deployment is required, but the provider is not selected. This section is a decision checklist, not a claim that hosting is configured.

Provider checklist

- Supports Java 21-compatible Spring Boot deployment.
- Provides or securely connects to MySQL.
- Allows environment variables/secret management and HTTPS.
- Provides useful application logs without exposing secrets.
- Has suitable persistence/free-tier limits and predictable availability during demo.
- Allows database export and restore; team understands sleep/cold-start limitations.

Environments

Environment	Purpose	Guidance
Local	Each teammate development	Document JDK/Maven/MySQL; local secrets ignored by Git.
Test	Automated/integration tests	Separate controlled test data/database.
Production/demo	Online faculty demonstration	Secrets via host config, HTTPS, approved schema setup, backup and logs.

Release checklist

1. Build from clean clone using documented Java/Maven versions.
2. Run unit/integration tests and fix release blockers.
3. Create/verify production schema using approved migration procedure.
4. Configure secrets outside source control and deploy.
5. Smoke-test registration, login, project creation, invites, tasks, archive and denied actions.
6. Export a backup and test restore.
7. Record URL, release commit, known limitations and demo instructions securely.

15. Timeline: 12–31 October 2026

Dates	Focus	Exit criteria
12 Oct	Approval/design lock and environment	Review blockers, repository setup, build skeleton, faculty questions, owners/dates.
13–18 Oct	Foundation	Build runs; MySQL connected; auth/security/profile/skills flows tested.
19–23 Oct	Discovery and teams	Search/matching, projects, invites, membership, leave/ownership rules integrated.
24–27 Oct	Tasks/portfolio/archive	Task status, progress, portfolio, archive enforcement integrated.
28–30 Oct	Stabilize, deploy, document	Critical tests, online demo, backup/restore, SRS traceability and rehearsal.
31 Oct	Submission	Required artifacts and demo ready per official checklist.

Daily cadence

- Short sync: done, next, blocker.
- Integrate and build daily; don't leave merge/integration until final week.
- Test each feature as it lands.
- Reserve 28–30 October for fixes, docs, deployment and rehearsal; reduce optional scope if behind.

16. Risks and open decisions

ID	Open item / risk	Why it matters	Suggested coordinator
B-01	Faculty rubric/UML requirements unknown.	May affect diagrams and grading artifacts.	Charan
B-02	Hosting provider and DB hosting unknown.	Blocks production configuration/deploy tests.	Manish + team
B-03	Self-declared student eligibility.	Status is not independently verified.	Team
B-04	Username normalization/change policy open.	Affects unique constraint and login.	Manish + Balanand
B-05	Manual password recovery procedure open.	Must prevent account takeover and password disclosure.	Balanand + Charan
B-06	Ownership transfer timeout/owner unavailable.	Could leave transfer pending or block departure.	Team
B-07	Leader waits if owner unavailable.	Potential operational deadlock accepted by current proposal.	Team approval
B-08	Former-member archived visibility unspecified.	Could expose unintended information.	Ephraim + Balanand
B-09	Production schema migration open.	Hibernate update alone is insufficient strategy.	Manish
B-10	Task due-date timezone/boundary open.	Affects overdue calculation.	Balanand + Manish
B-11	Audit visibility/retention open.	Affects privacy and operations.	Charan + team
B-12	Official submission list unverified.	Could miss required artifact.	Charan
B-13	FR/use-case/test traceability not finalized.	Can cause inconsistent IDs/documents.	Charan

For each blocker, record the exact question, responsible person, due date and approved fallback if any. Do not silently resolve open questions by implementing the easiest option.

17. SRS structure and consistency

Faculty section	SkillForge content
1. Introduction	Purpose, scope, audience, definitions, references, document overview.
2. Overall Description	Product perspective/functions, user classes, environment, constraints, assumptions/dependencies.
3. Specific Requirements	FR-01…FR-09 in testable 'shall' form, interfaces, use cases and business rules as template requires.
4. Non-functional Requirements	Security, usability/accessibility, browser support, performance at demo scale, availability, audit and backup.
5. Other Requirements	Database/deployment constraints and applicable extra requirements; do not invent.
6. Activity Log	Dated record of actual activities, decisions and document revisions.

Consistency rules

- Every FR has a unique ID and acceptance criteria.
- Use-case IDs stay stable across SRS and diagrams.
- UML names match actual code or are labelled proposed.
- Tests link to FR/use case and show actual result, not just planned status.
- Changes update SRS, diagrams, tests and code as appropriate.
- Activity Log records actual work only.
- This handbook is a planning reference, not a replacement for the faculty SRS template.

18. Definition of done and daily checklist

A feature is done only when all applicable items are satisfied:

- Behavior matches an approved requirement and correct role.
- Server-side validation and authorization are implemented.
- Success, empty, invalid and unauthorized states are handled.
- Relevant tests are added and executed; results recorded.
- Changes are reviewed/committed and shared main branch builds.
- UI uses agreed routes/DTO contracts and does not expose unapproved data.
- Database setup is reproducible and secrets are not committed.
- SRS, traceability, diagrams and known limitations are updated when behavior changes.
- Feature has been tested in integrated application, not only in isolation.

Daily update template

```
Name:
Date:
Completed (commit/PR/artifact):
Next task:
Blocker / decision needed:
Files or APIs changed:
Tests run and results:
Handoff required from/to:
```

Start checklist for 12 October

- Verify all four can clone/push to GitHub.
- Check Java 21, Maven, VS Code tooling and MySQL access.
- Choose compatible Spring Boot version and create initial build.
- Agree package/entity/enum/route/DTO/service contracts.
- Assign each FR and technical deliverable an owner and target date.
- Ask faculty/coordinator for rubric, UML and official deliverable checklist.
- Assign a hosting decision owner/date.
- Do not parallelize feature work until the shared skeleton builds for everyone.

End of handbook. Update this document when the team explicitly changes or approves decisions. Proposed choices must remain labelled until approved.